CMC UNIVERSITY

Aspire to Inspire the Digital World

Machine Learning and Data Mining

Faculty of Information and Communication Technology

2.1. Convolutional Neural Network

2.2 Recurrent Neural Network

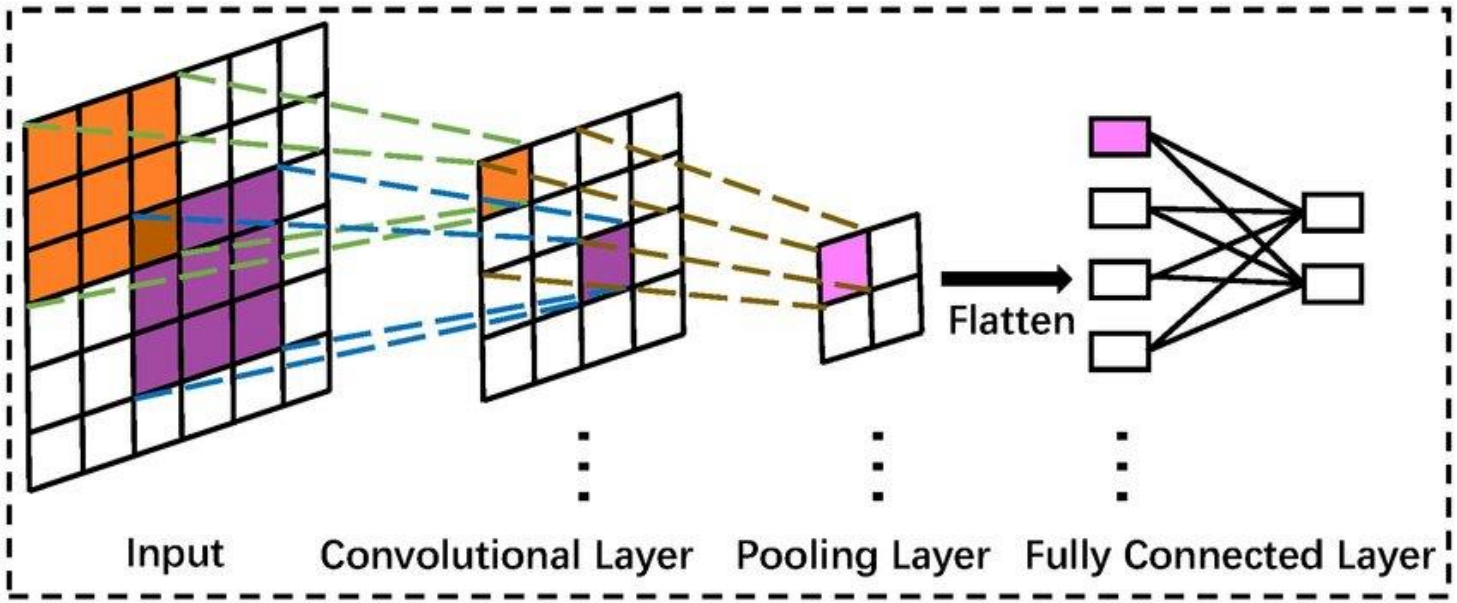
2.3 Long-Short Term Memory

2.4 Optimization techniques

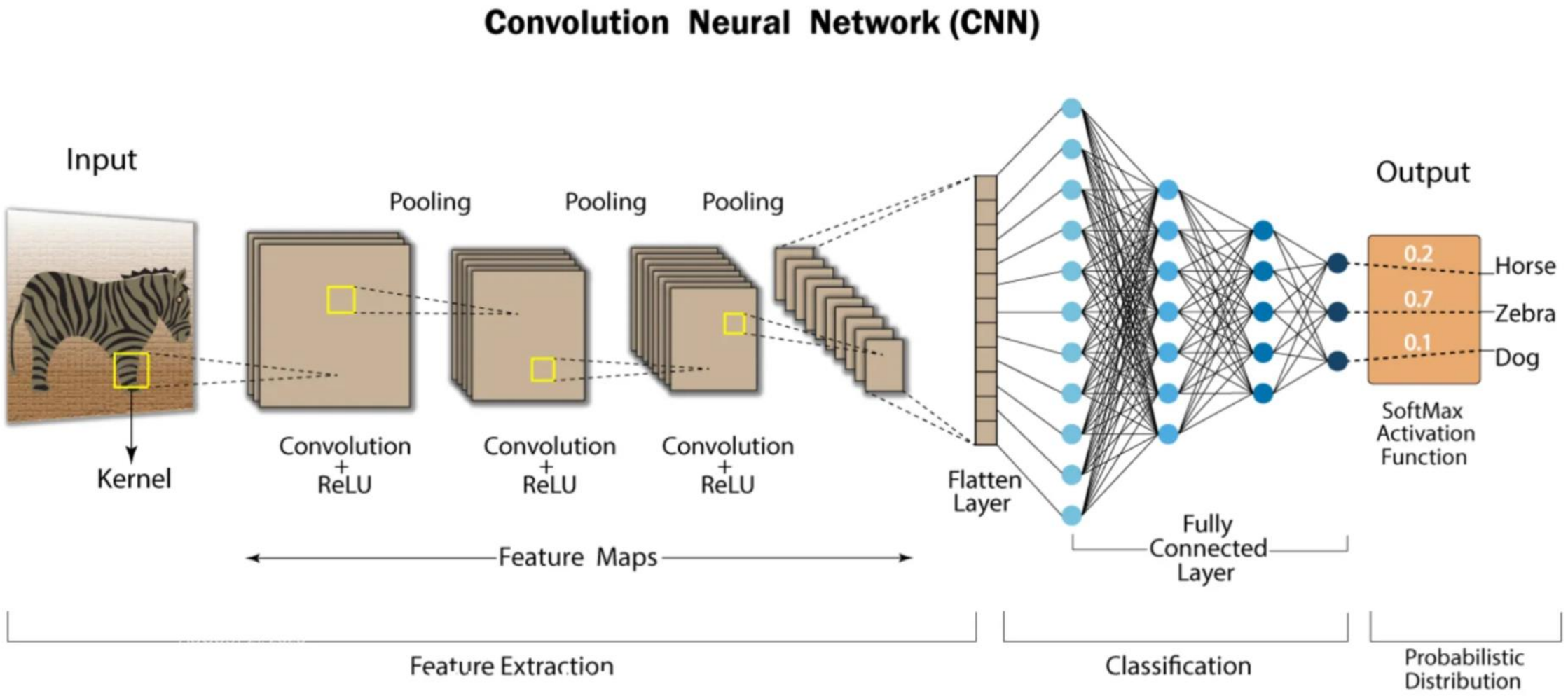
Convolutional Neural Network

Convolutional Neural Networks

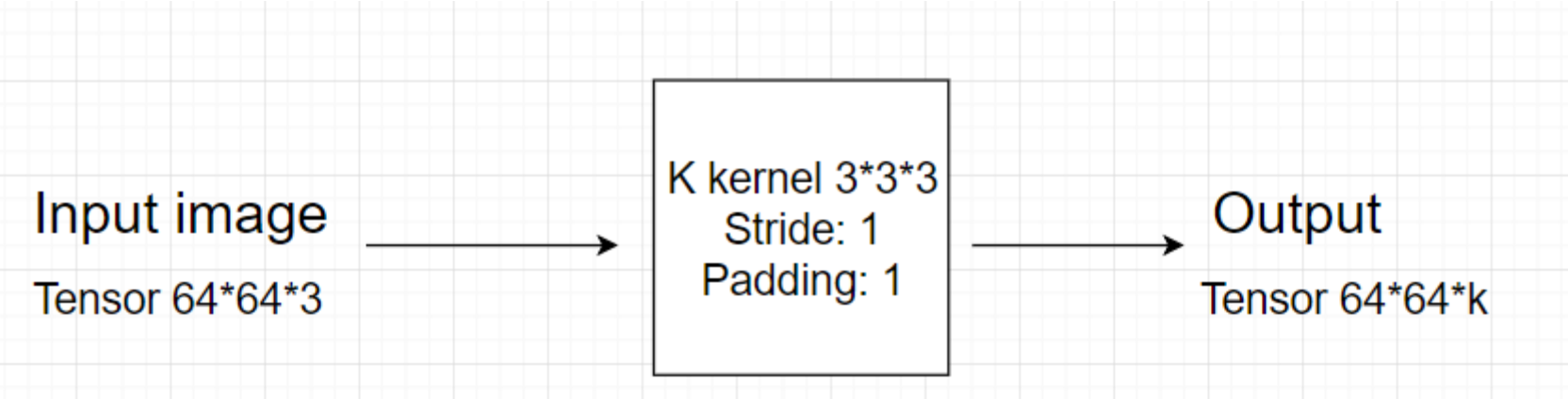
- A type of artificial neural network
- Specialized in processing grid-like structured data
- Best suited for image classification, object detection, image segmentation, and speech recognition
- CNN typically consists of 3 layers:
- Convolutional layer
- Pooling layer
- Fully connected layer



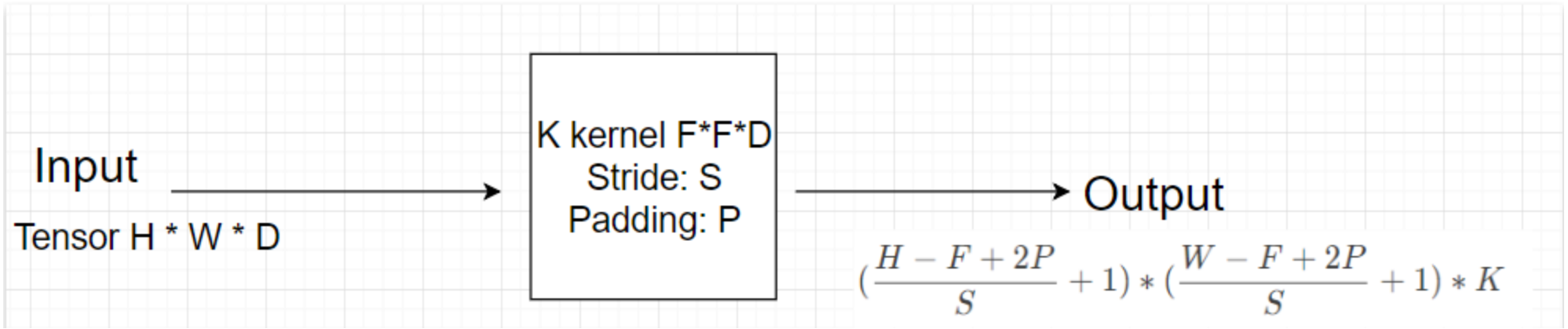
Convolutional Neural Networks



- A math operation is called a convolution
- Use the sliding window function for the pixel matrix (kernel or filter)
- Can detect features such as edges, textures, or shapes
- Each filter is used to identify a specific pattern from the image: find lines, curves, etc.

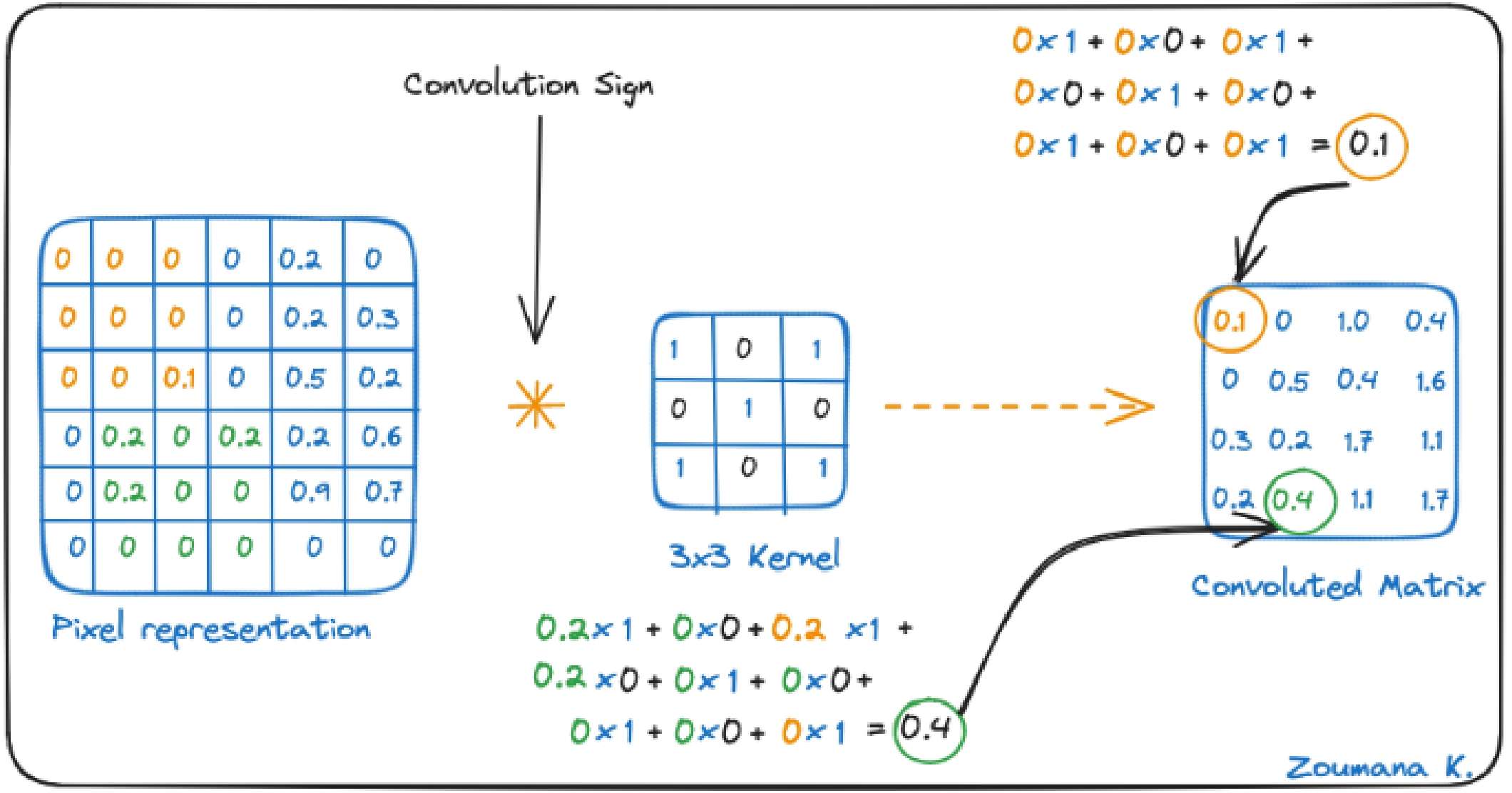


- The output of the convolutional layer using the activation function becomes the input of the next convolutional layer.
- Number of parameters (bias, K kernel): (F*F*D +1)*K



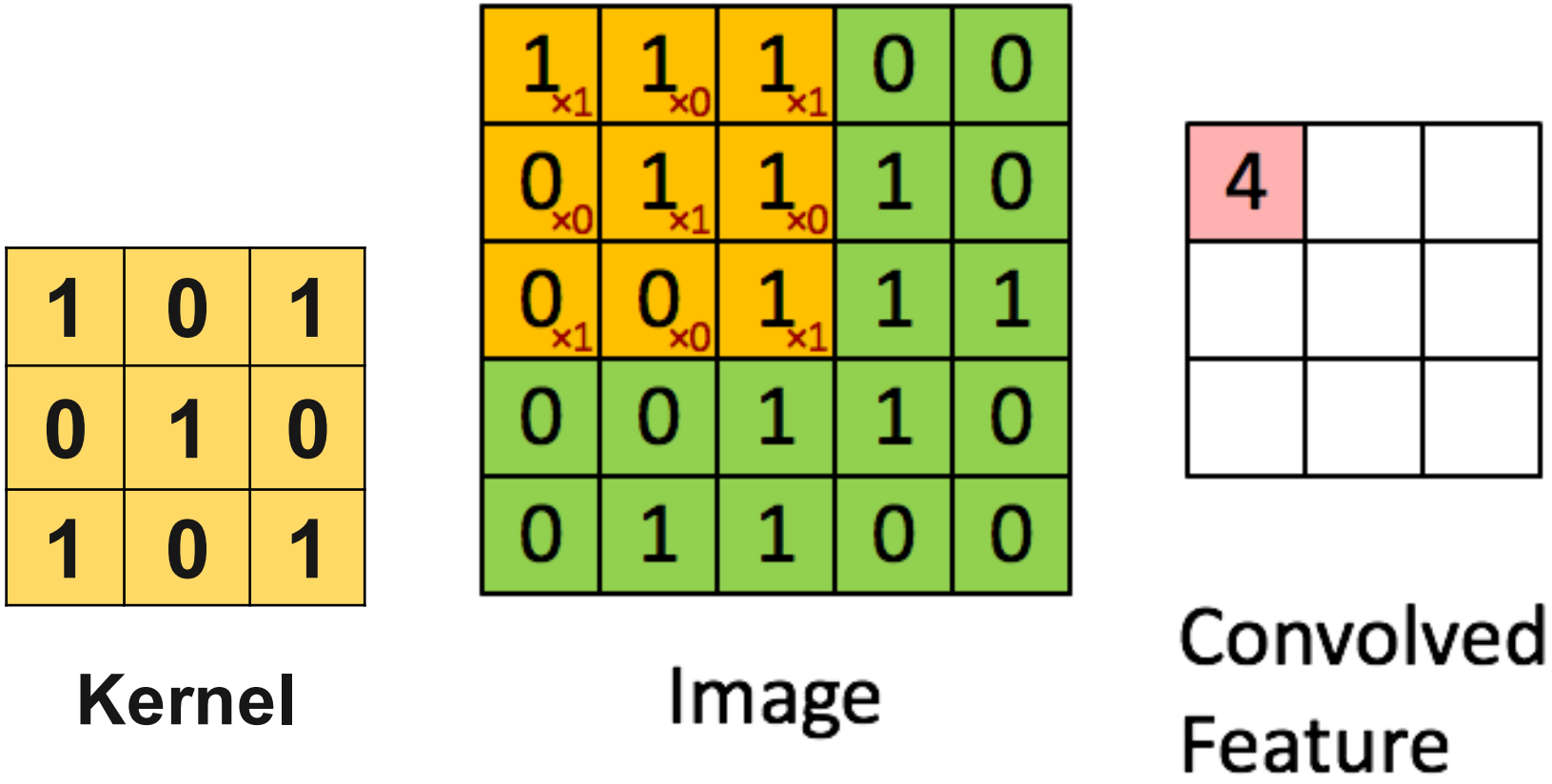
Convolution operation:

1. Apply the multiplier matrix from the top left corner to the right.
2. Perform elemental multiplication.
3. Calculate the total value of the multiplications.
4. The resulting value corresponds to the first value (top left corner) in the convolutional matrix.
5. Move multiplied by the size of the sliding window.
6. Repeat steps 1 through 5 until the image matrix is fully calculated.



Apply convolution using stride 1 with a 3x3 kernel

(https://datacamp.com)



Convolutional Calculation

Convolutional Neural Networks - Convolution layers

1 Apply the multiplier matrix from the top left corner to the right.

Image matrix

1	2	3	4	5
6	7	8	9	10
11	12	13	14	15
16	17	18	19	20
21	22	23	24	25

Multiplier matrix (3×3 kernel)

1	0	−1
1	0	−1
1	0	−1

2 Perform elemental multiplication.

Element-wise multiplication

1	2	3
6	7	8
11	12	13

⊗

1	0	−1
1	0	−1
1	0	−1

=

1	0	−3
6	0	−8
11	0	−13

3 Calculate the total value of the multiplications.

Sum all elements

1	0	−3
6	0	−8
11	0	−13

Total sum = −20

4 The resulting value corresponds to the first value (top left corner) in the convolutional matrix.

Image matrix

1	2	3	4	5
6	7	8	9	10
11	12	13	14	15
16	17	18	19	20
21	22	23	24	25

Output (convolution result) matrix

−20		

5 Move multiplied by the size of the sliding window.

Slide the 3×3 window to the right.

1	2	3	4	5
6	7	8	9	10
11	12	13	14	15
16	17	18	19	20
21	22	23	24	25

Output matrix (update)

−20	−20	

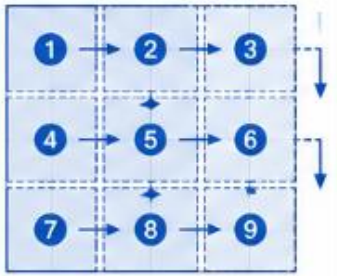
Next positions...

6 Repeat steps 1 through 5 until the image matrix is fully calculated.

Final output (3×3)

−20	−20	−20
−60	−60	−60
−100	−100	−100

Scanned positions



3×3 Multiplier matrix (kernel)

Sliding window (size 3×3)

Output matrix size
(5−3+1) × (5−3+1) = 3×3

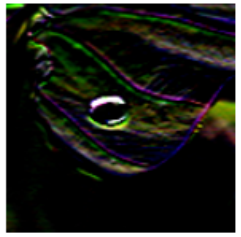
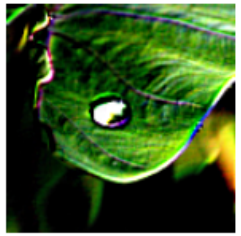
General formula:

Output size = (M − K + 1) × (N − K + 1)

M, N: image height, width; K: kernel size

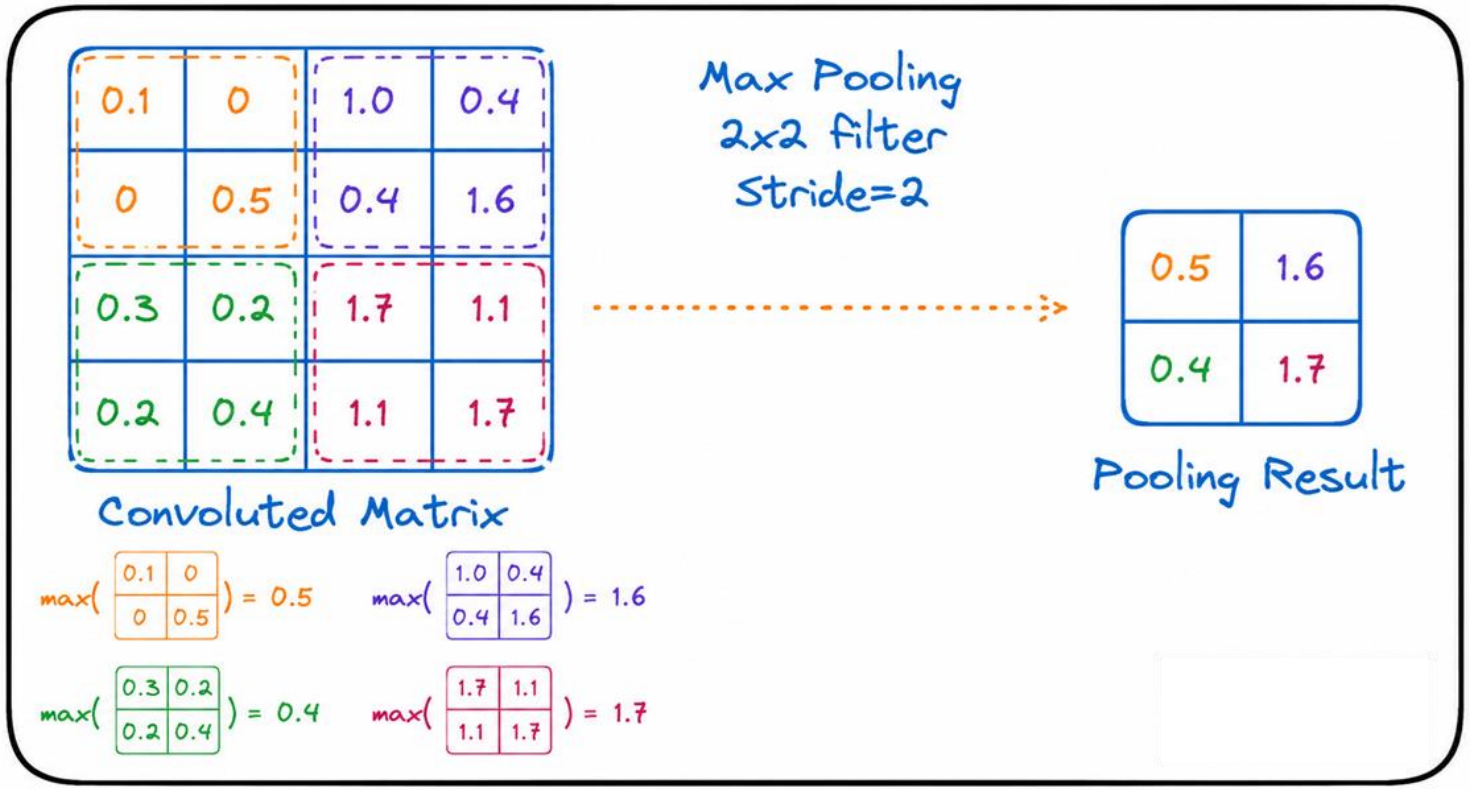
Convolutional Neural Network – Kernel samples

CMC UNIVERSITY
Aspire to Inspire the Digital World

	Identity (no filtering)	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$		Sharpening	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 9 & -1 \\ -1 & -1 & -1 \end{bmatrix}$
	Binomial blur filter	$\frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$		Gradient X	$\begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix}$
	Gradient Y	$\begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$			
	Emboss	$\begin{bmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}$			

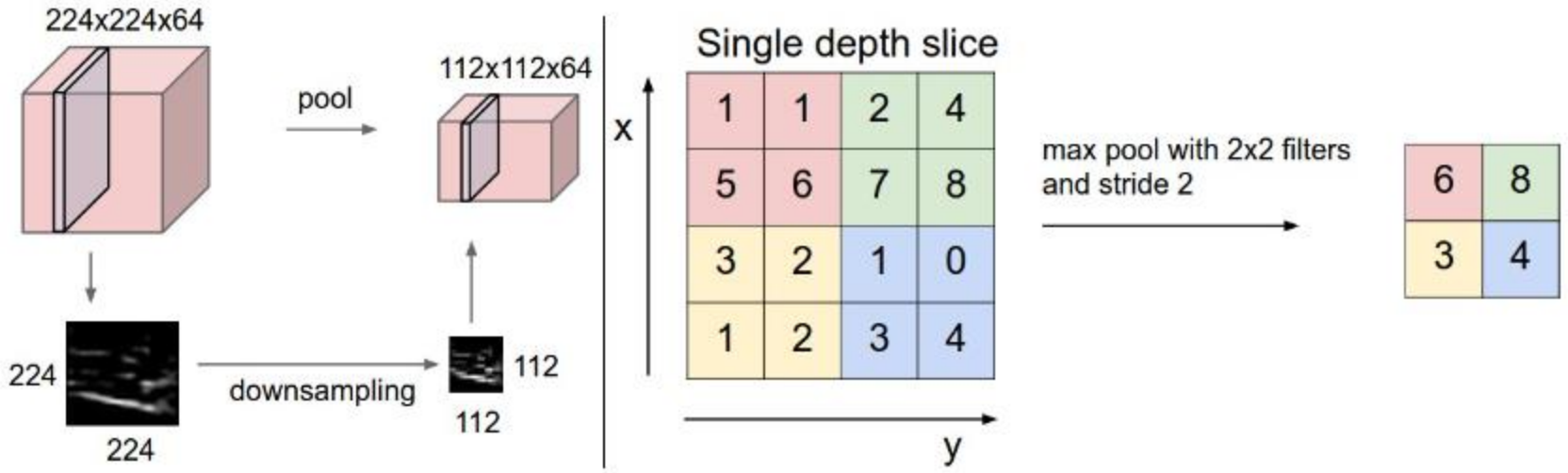
https://www.gsn-lib.org/

- Reduced feature space by convolutional layers
- Reduce the size of the space while retaining important information



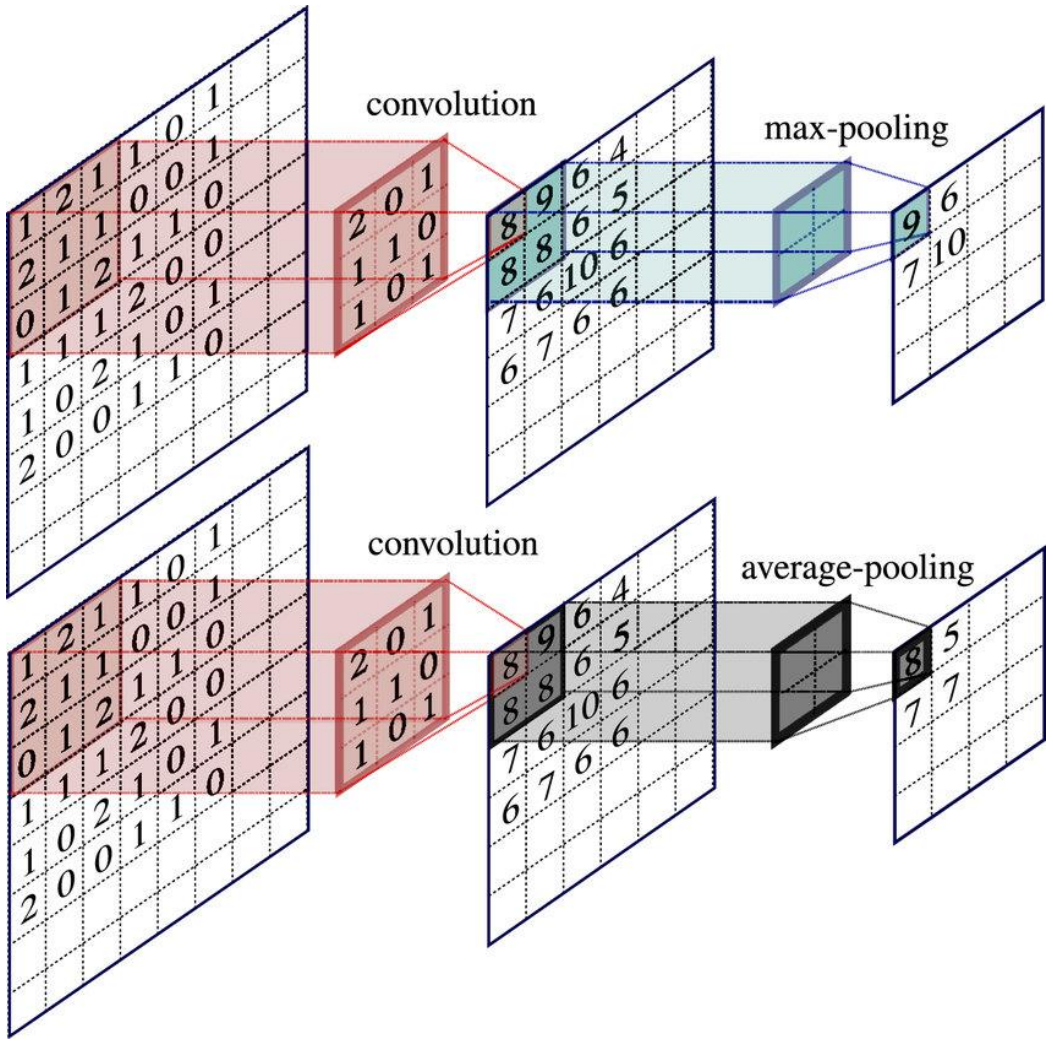
Apply Max Convolution with stride = 2 and 2x2 filter

The most common compound functions that can be applied are:
Max pooling: the greatest value of a typical space
Sum pooling: sum of all the values of a characteristic space
Average pooling: the average of all values.



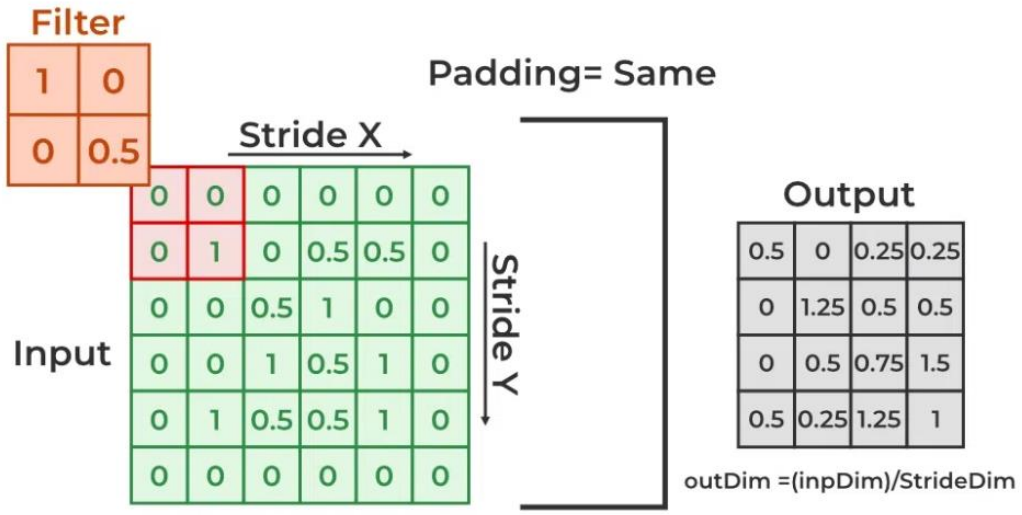
- ✓ *reduce the memory while train model.*
- ✓ *Feature space becomes smaller*
- ✓ *Mitigate overfitting*

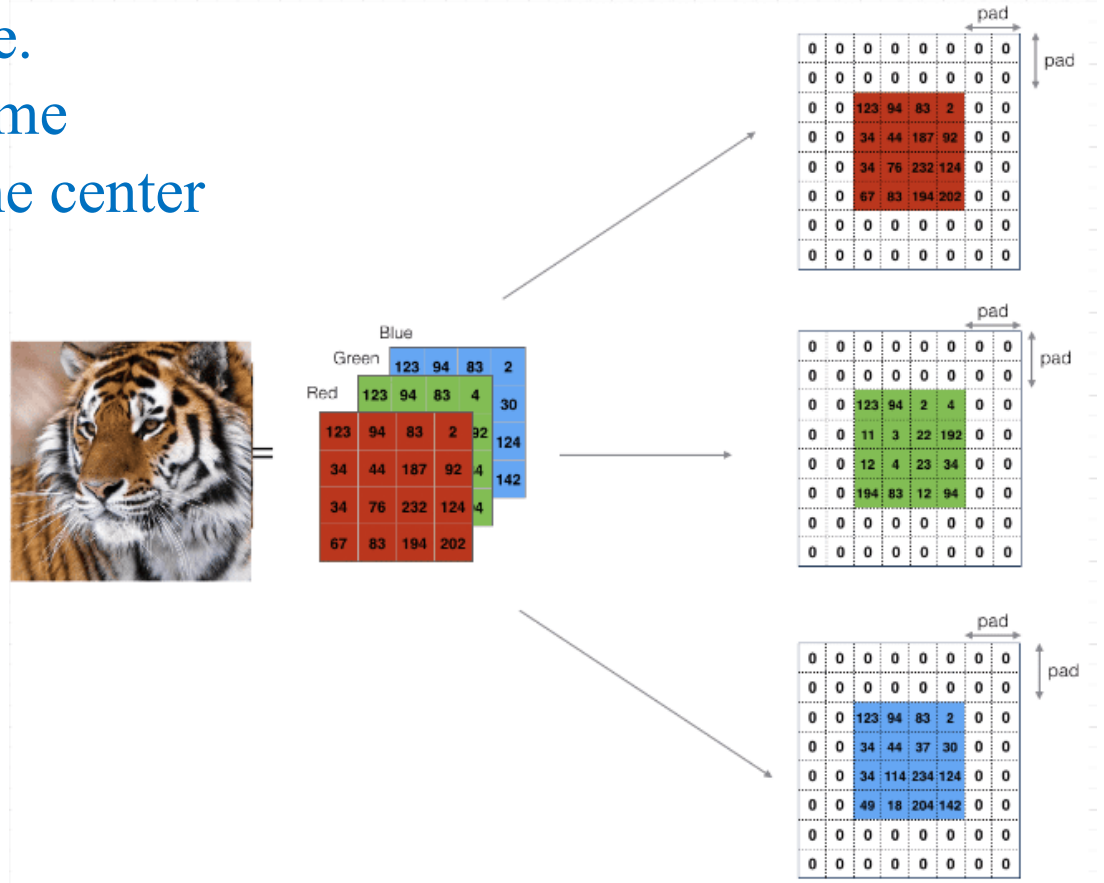
Compounding operations: max pooling and average pooling.



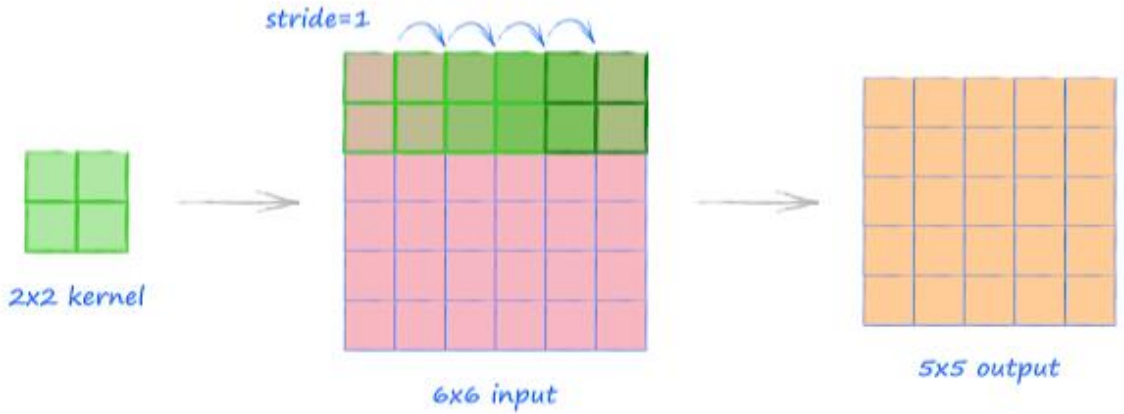
In some models, people use a convolution layer with stride >1 to reduce the data size instead of the pooling layer.

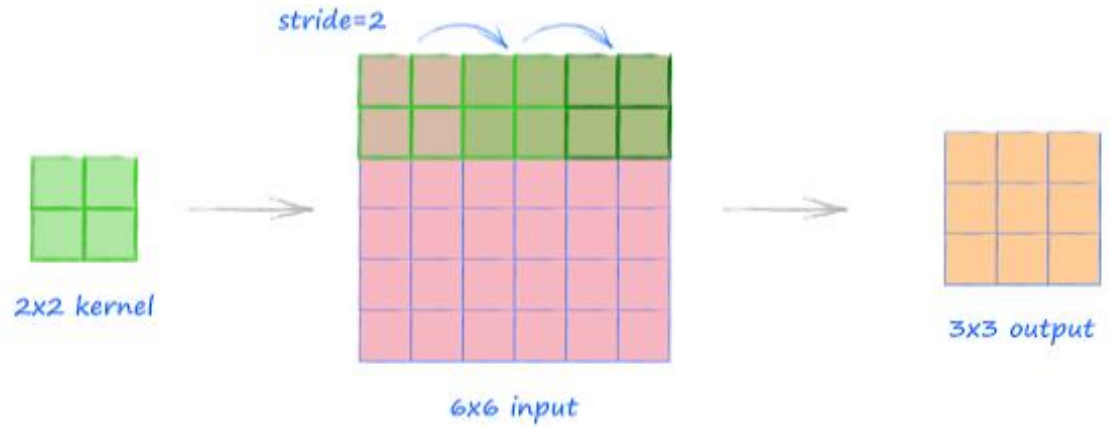
- Stride: The number of pixels that the filter/core moves through
  - Stride = 1: the filter moves one pixel at a time, while Stride = 2: moves two pixels at a time,
  - Larger Stride Results in Smaller Output Dimensions
- Padding: maintains the spatial size of the input image. It adds additional pixels (usually zeros) around the border of the input image.
  - Don't want the photo to shrink every time
  - The edges of the image use less than the center

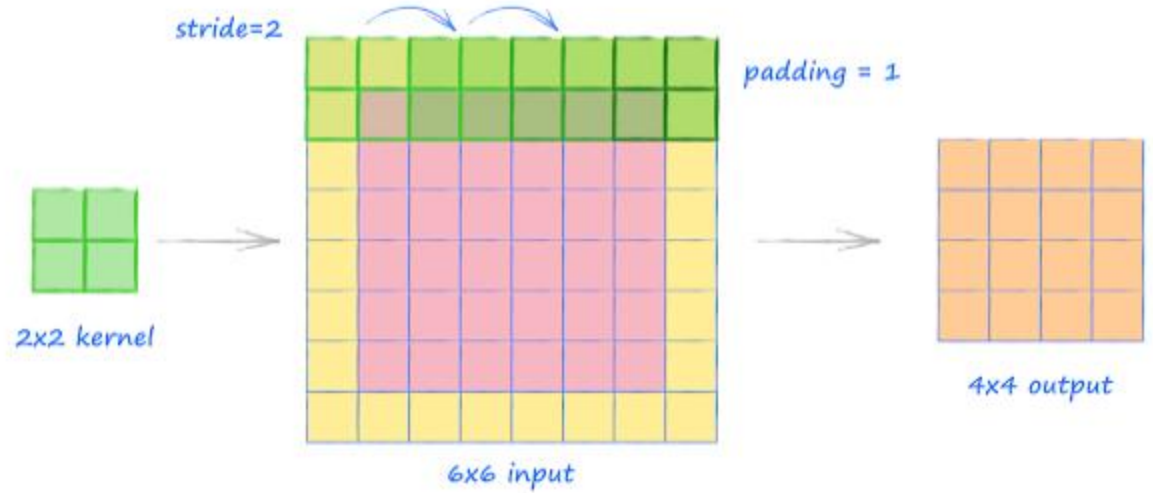




Convolutional Neural Networks - Stride and padding



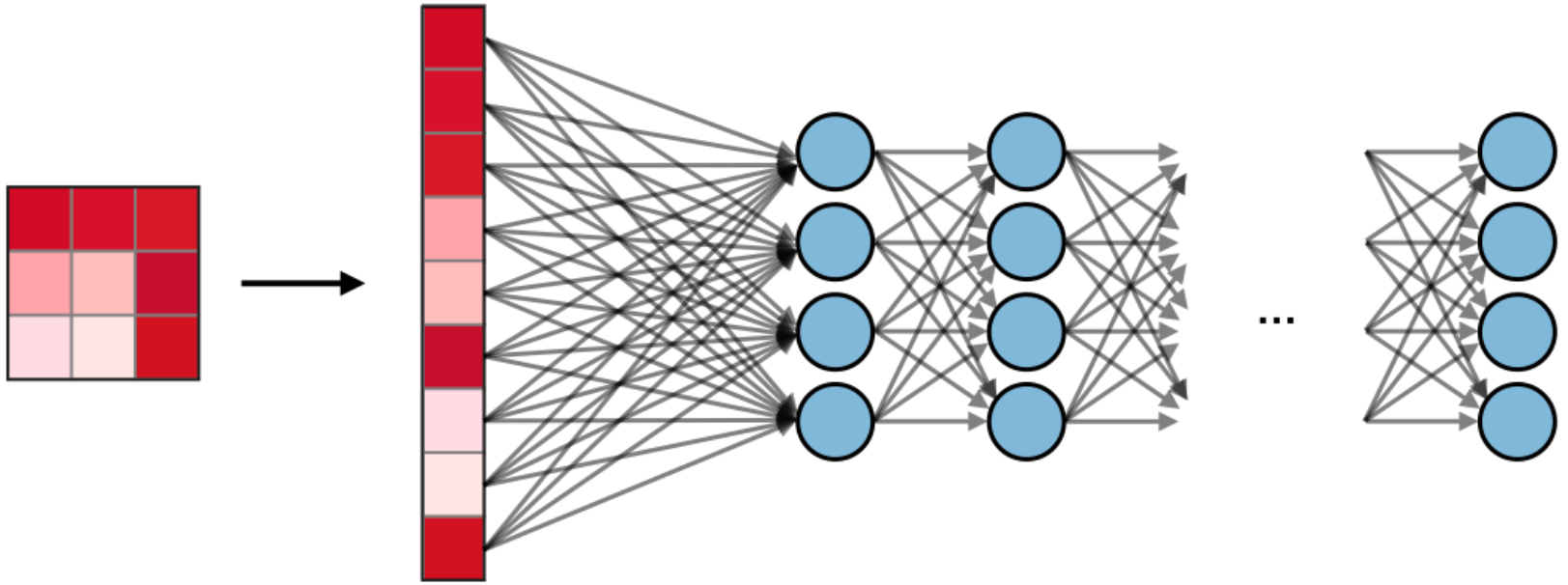




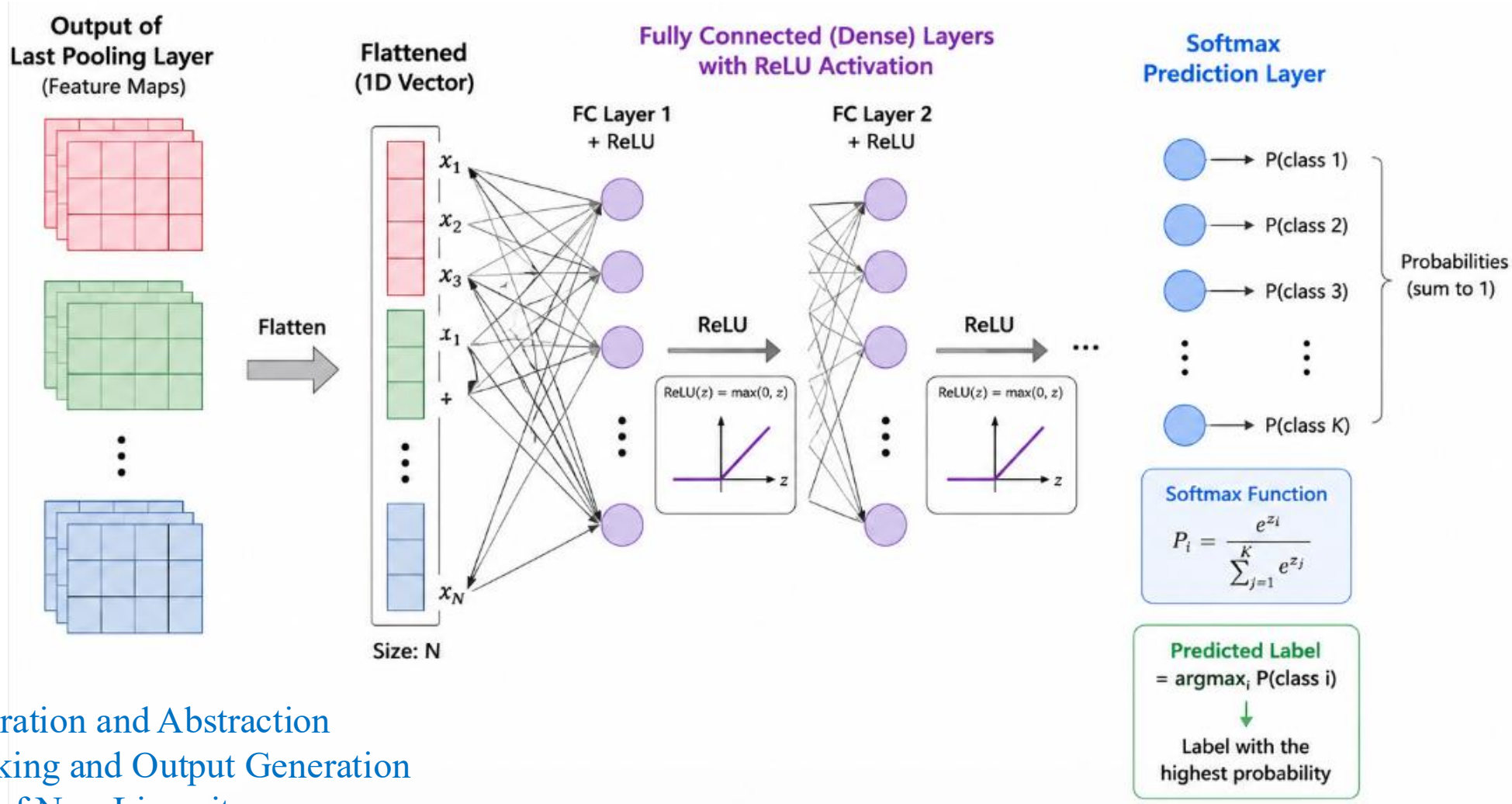
$$\text{output size} = \left(\frac{\text{input size} + 2 \times \text{padding} - \text{kernel size}}{\text{stride}} \right) + 1$$

- input size: Size of the input feature map
- 2 x padding: Padding added to both sides
- kernel size: Size of the pooling/convolution window
- stride: Step size of the sliding window
- + 1: Add 1

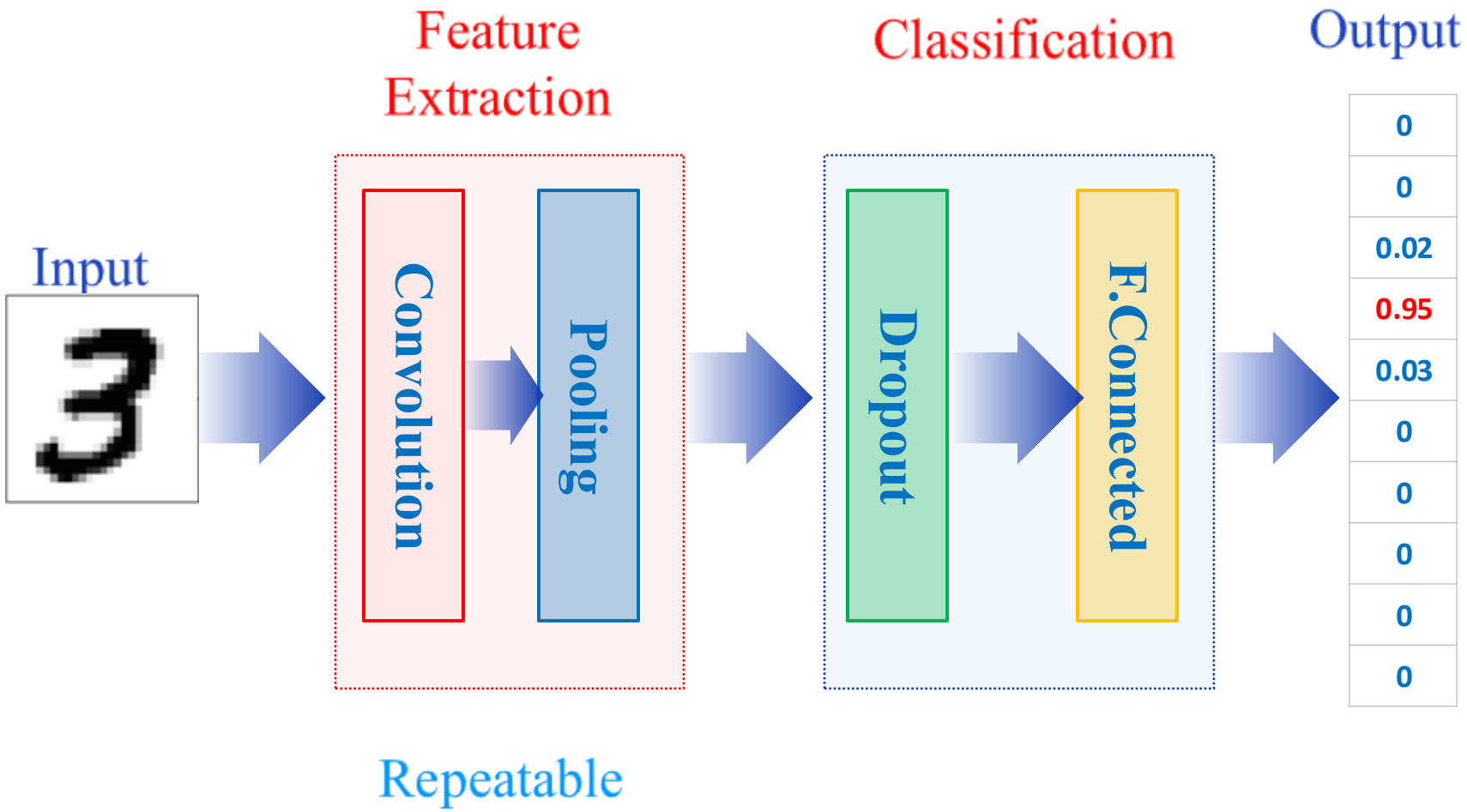
The fully connected layer (FC or Dense layer) operates on a flattened input where each input is connected to all neurons. If present, FC layers are usually found towards the end of CNN architectures and can be used to optimize objectives such as class scores.



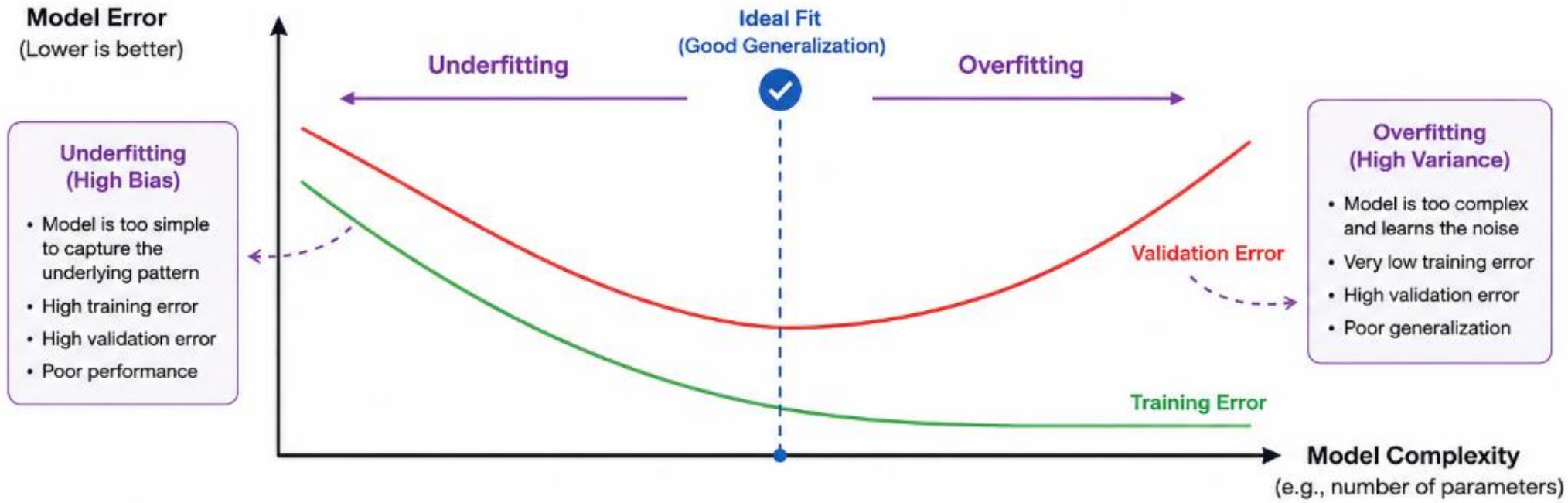
Convolutional Neural Networks – Fully connected layers



FC layer:
+ Feature Integration and Abstraction
+ Decision Making and Output Generation
+ Introduction of Non-Linearity
+ Universal Approximation


Feature Extraction
Classification
Output
Input
Convolution
Pooling
Dropout
F.Connected
0
0
0.02
0.95
0.03
0
0
0
0
0
Repeatable

Convolutional Neural Networks – Overfitting and Underfitting



Underfitting (High Bias)

The model is too simple to learn the underlying patterns.

Both training and validation errors are high.

Ideal Fit (Good Generalization)

The model captures the underlying patterns without learning the noise.

Validation error is minimized and close to training error.

Overfitting (High Variance)

The model is too complex and learns the noise in the training data.

Training error is very low, but validation error is high.

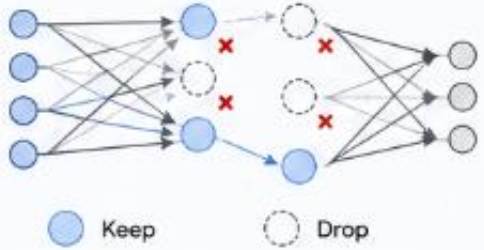
Choose a model complexity that minimizes validation error and achieves good generalization to new data

7 Strategies to mitigate Overfitting in CNN

CMC UNIVERSITY
Aspire to Inspire the Digital World

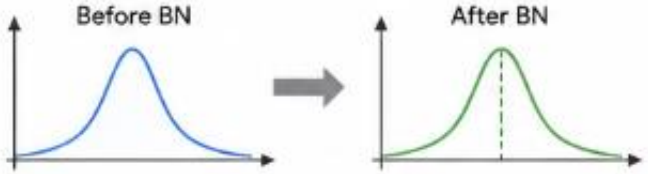
1 Dropout

Randomly drops (sets to zero) a fraction of neurons during training, preventing co-adaptation and improving generalization.



2 Batch Normalization

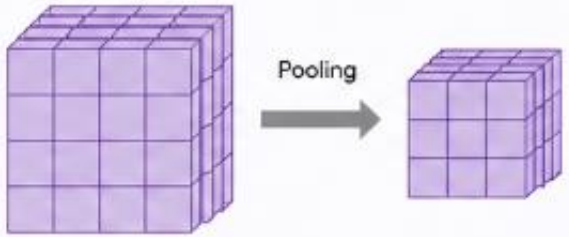
Normalizes layer inputs within a mini-batch to stabilize and accelerate training, which also acts as a regularizer.



More stable distribution → Better generalization

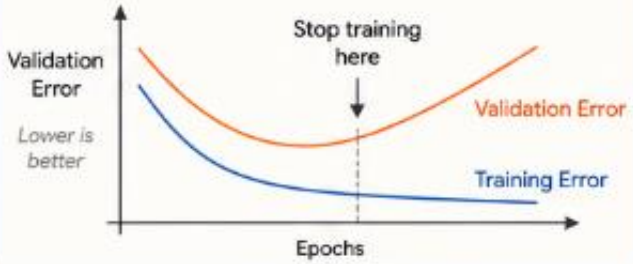
3 Pooling Layers

Reduces the spatial size of feature maps, keeping the most important information and improving translation invariance.



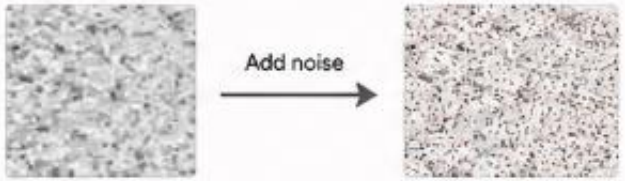
4 Early Stopping

Monitors validation performance and stops training when it starts to degrade.



5 Noise Injection

Adds small random noise to inputs or activations to make the model more robust to variations.



Encourages model robustness

6 L1 and L2 Regularizations

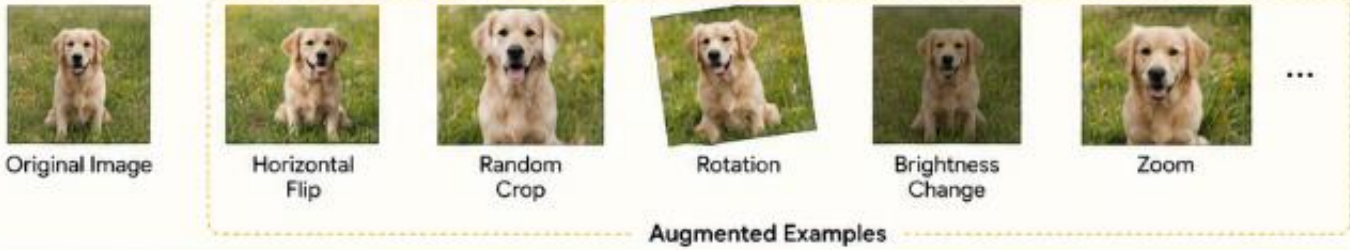
Adds a penalty on large weights to the loss function, encouraging simpler models.

L2 (Ridge)
Penalizes large weights heavily

L1 (Lasso)
Encourages sparsity (some weights → 0)

7 Data Augmentation

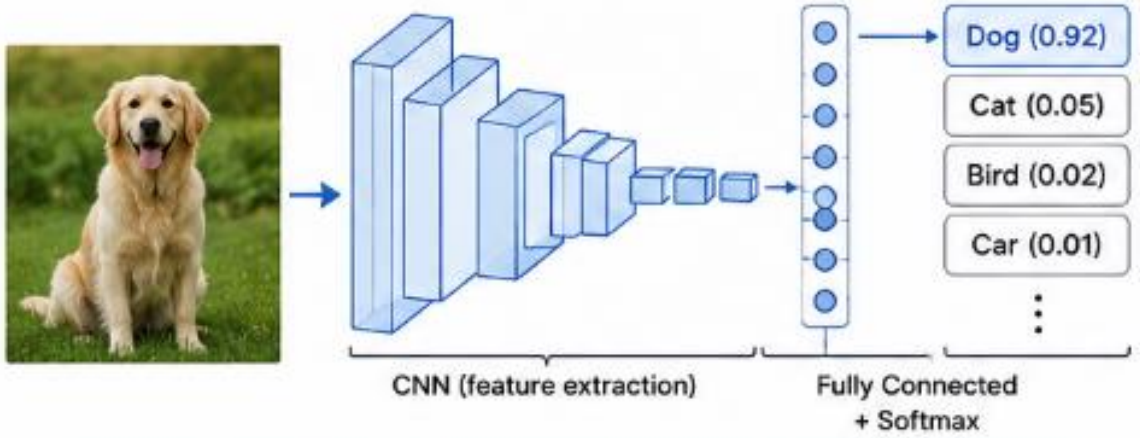
Artificially increases the diversity of training data by applying random transformations, helping the model generalize better to unseen data.



Convolutional Neural Networks – Applications

1 Image Classification

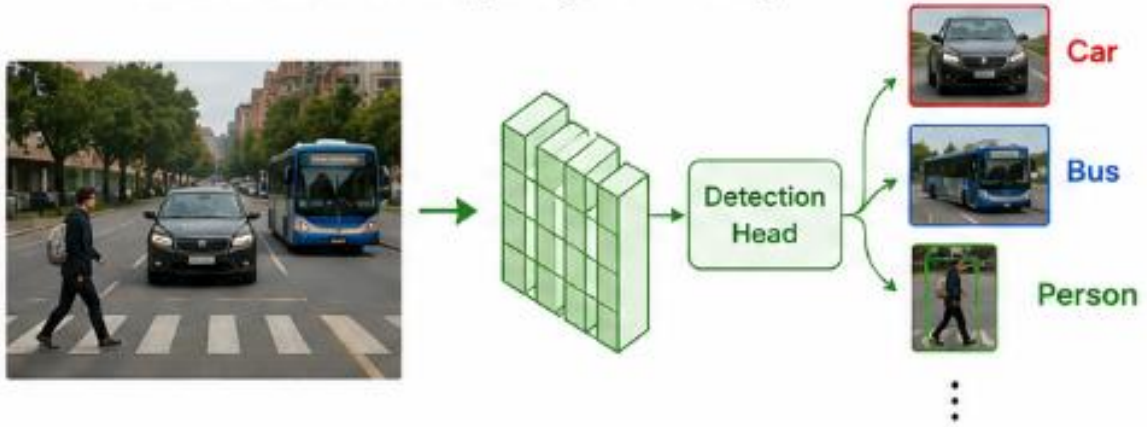
Predicts the class label of an entire image.



Example: Classify an image as dog, cat, bird, etc.

2 Object Detection

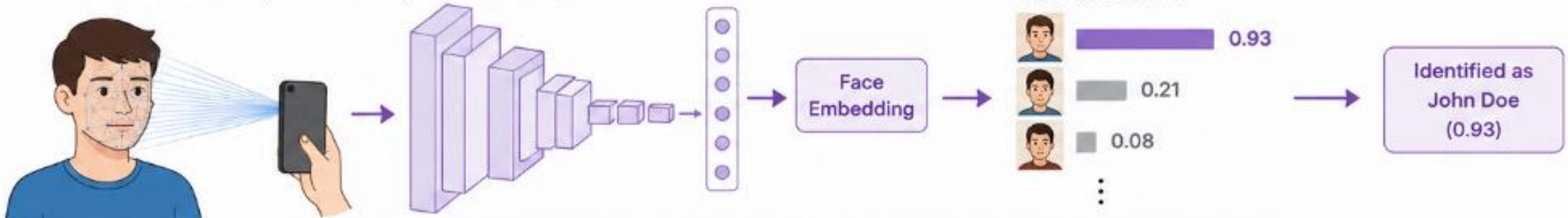
Detects and localizes multiple objects in an image.



Example: Detect and locate objects like cars, people, buses, etc.

3 Facial Recognition

Identifies or verifies a person's identity from a face image.



Example: Unlock phone, access control, attendance systems, etc.

Popular CNN Architectures

- LeNet-5 (1998): One of the first CNNs, designed for handwritten digit recognition.
- AlexNet (2012): Won the ImageNet competition and popularized deep CNNs with GPU training.
- VGGNet (2014): Demonstrated that deeper networks with small 3x3 filters improve accuracy.
- GoogLeNet/Inception (2014): Introduced inception modules with parallel filter sizes for multi-scale feature extraction.
- ResNet (2015): Introduced skip connections, enabling training of networks with 100+ layers.
- EfficientNet (2019): Used compound scaling to balance network depth, width, and resolution.
- ConvNeXt (2022): A modernized CNN design that competes with Vision Transformers.
- …

https://www.kaggle.com/discussions/general/255114

Prerequisite: Tensorflow, Keras, Matplotlib

TensorFlow: An **open-source framework** developed by **Google Brain**.

- **Flexible architecture**: Simple feedforward networks to complex deep learning architectures.
- **High-Level APIs**: APIs like **Keras**, **tf.keras**, and **TensorFlow Estimators**.
- **Distributed training**: Supports multiple CPUs, GPUs, and machines.

- **TensorBoard**: A visualization toolkit.
- **Real-world deployment**: Used for production-level applications.

Keras: A high-level neural network API, enabling **quick experimentation in deep learning**.

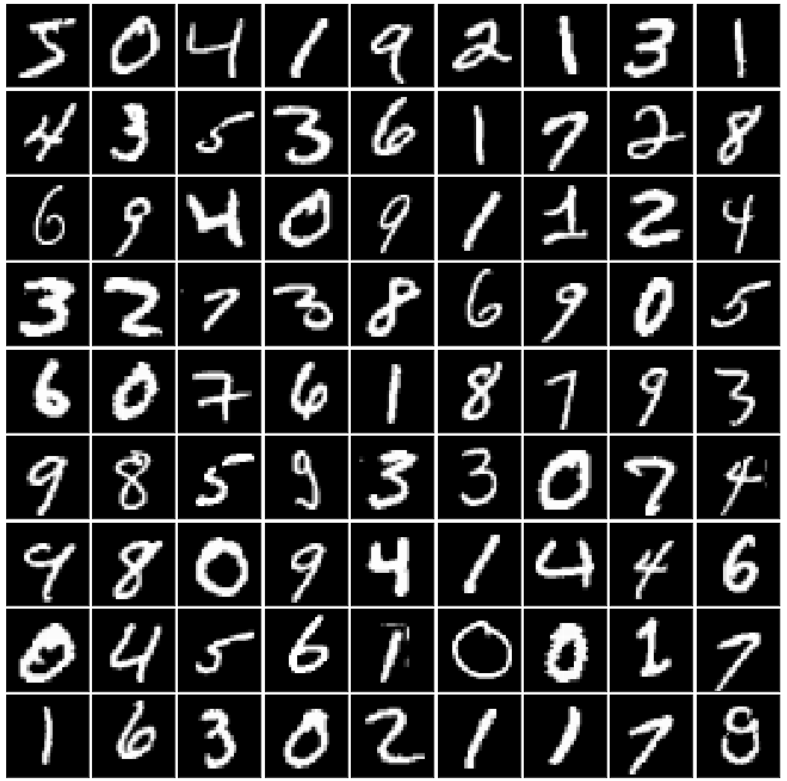
- **User-friendly interface**, **Flexible**, **Integrated with Backend Engines** (like TensorFlow), and **Scalable**.

Matplotlib: A comprehensive library for creating **plots**, **charts**, **histograms**, and other **visual representations** of data.

Prerequisites:Keras, Matplotlib

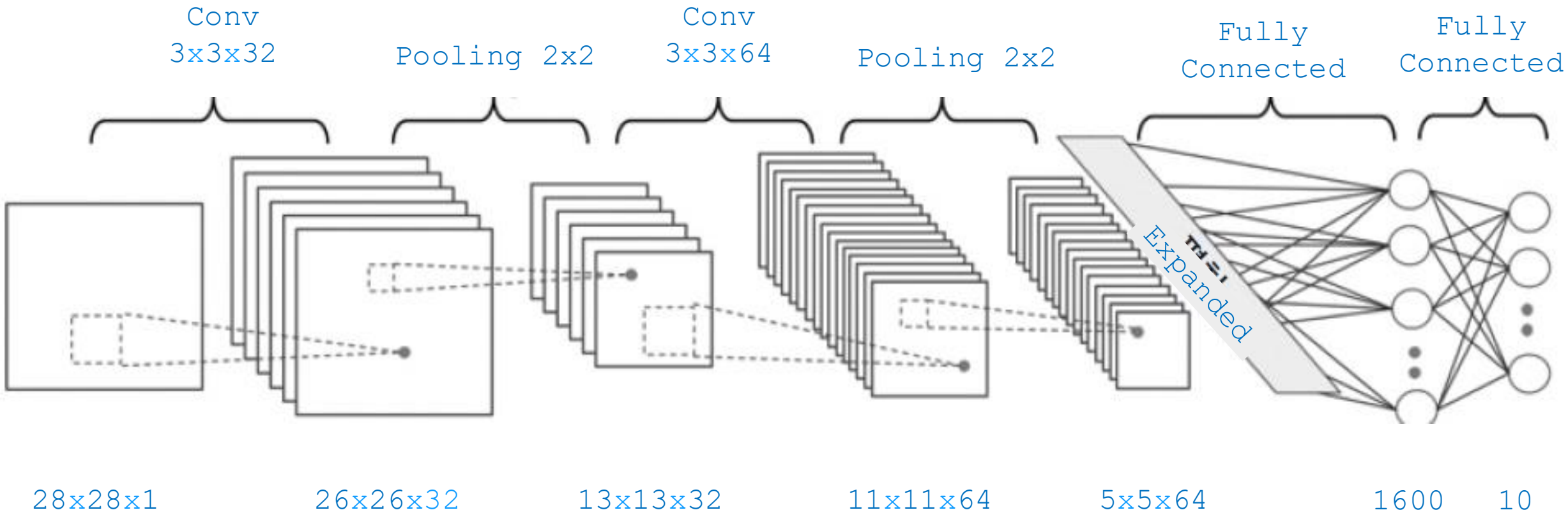
MNIST - A standard dataset used in training, online courses, and academic research projects.

- Handwritten digits
- Size: 28x28 pixels
- Training data: 60,000 images
- Test data: 10,000 images



CMC UNIVERSITY
Aspire to Inspire the Digital World

MNIST - CNN



Code:

```
# Load library
import numpy as np
import keras
from keras import layers
import matplotlib.pyplot as plt
```

```
num_classes = 10
input_shape = (28, 28, 1)
```

```
# Load the data and split it between train and test sets
(x_train, y_train), (x_test, y_test) =
keras.datasets.mnist.load_data()
```

Convolutional Neural Networks – MNIST Sample

```
# Scale images to the [0, 1] range
x_train = x_train.astype("float32") / 255
x_test = x_test.astype("float32") / 255
```

```
# Make sure images have shape (28, 28, 1)
x_train = np.expand_dims(x_train, -1)
x_test = np.expand_dims(x_test, -1)
```

```
# Check data shape
print("x_train shape:", x_train.shape)
print(x_train.shape[0], "train samples")
print(x_test.shape[0], "test samples")
```

Convolutional Neural Networks – MNIST Sample

```
# Check data shape
print("x_train shape:", x_train.shape)
print(x_train.shape[0], "train samples")
print(x_test.shape[0], "test samples")
```

```
>>> print(x_train.shape[0], "train samples")
60000 train samples
>>> print(x_test.shape[0], "test samples")
10000 test samples
>>>
```

```
# convert class vectors to binary class matrices
y_train = keras.utils.to_categorical(y_train, num_classes)
y_test = keras.utils.to_categorical(y_test, num_classes)
```

Convolutional Neural Networks – MNIST Sample


```
model = keras.Sequential(
    [
        keras.Input(shape=input_shape),
        layers.Conv2D(32, kernel_size=(3, 3), activation="relu"),
        layers.MaxPooling2D(pool_size=(2, 2)),
        layers.Conv2D(64, kernel_size=(3, 3), activation="relu"),
        layers.MaxPooling2D(pool_size=(2, 2)),
        layers.Flatten(),
        layers.Dropout(0.5),
        layers.Dense(num_classes, activation="softmax"),
    ]
)
```

```
model.summary()
```

```
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dropout (Dropout)	(None, 1600)	0
dense (Dense)	(None, 10)	16,010

```
Total params: 104,480 (408.13 KB)
Trainable params: 34,826 (136.04 KB)
Non-trainable params: 0 (0.00 B)
```

Convolutional Neural Networks – MNIST Sample

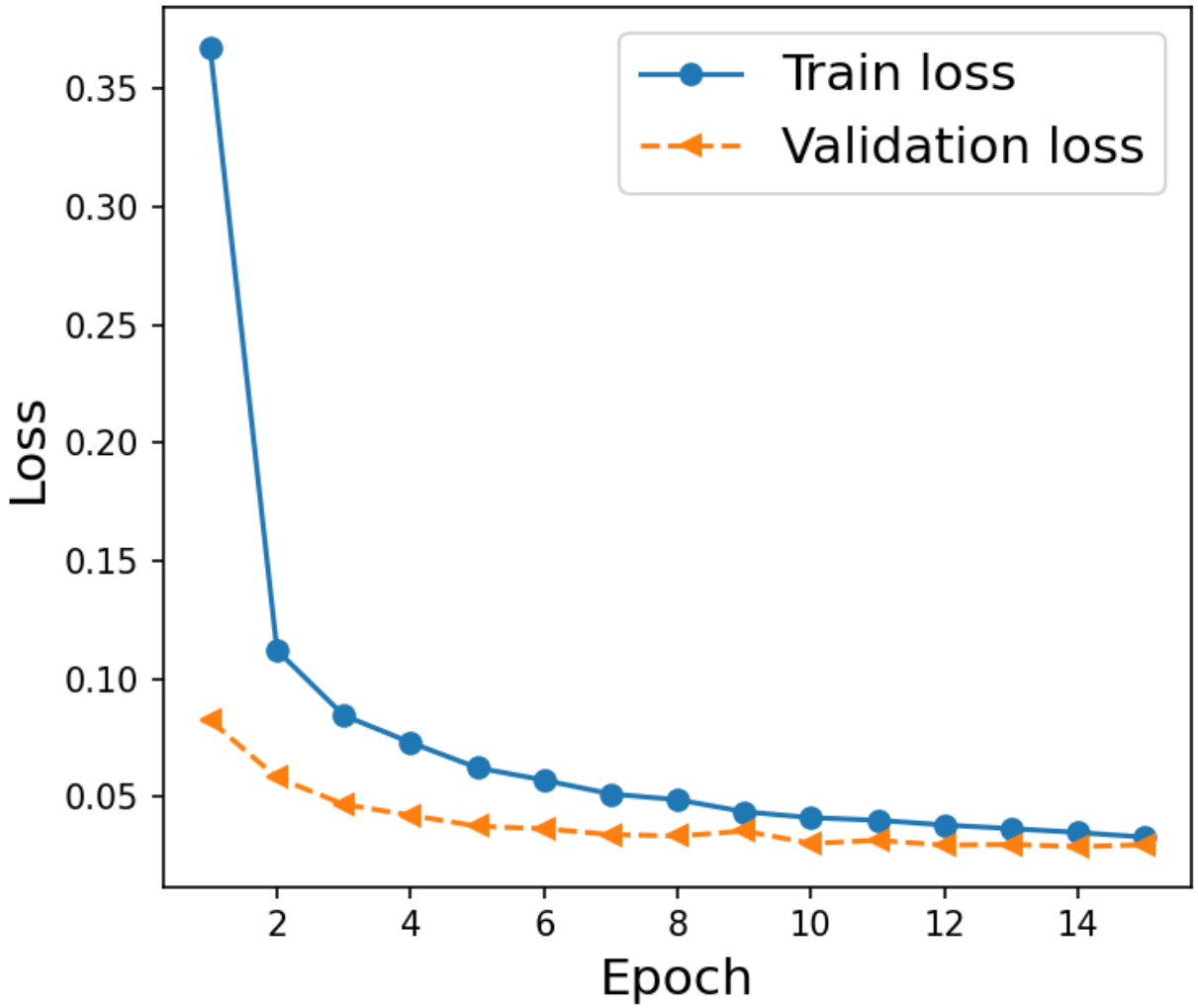
```
# Set parameters for the model and run
model.compile(loss="categorical_crossentropy", optimizer="adam", metrics=["accuracy"])
history = model.fit(x_train, y_train, batch_size=128, epochs=15, validation_split=0.1)
```

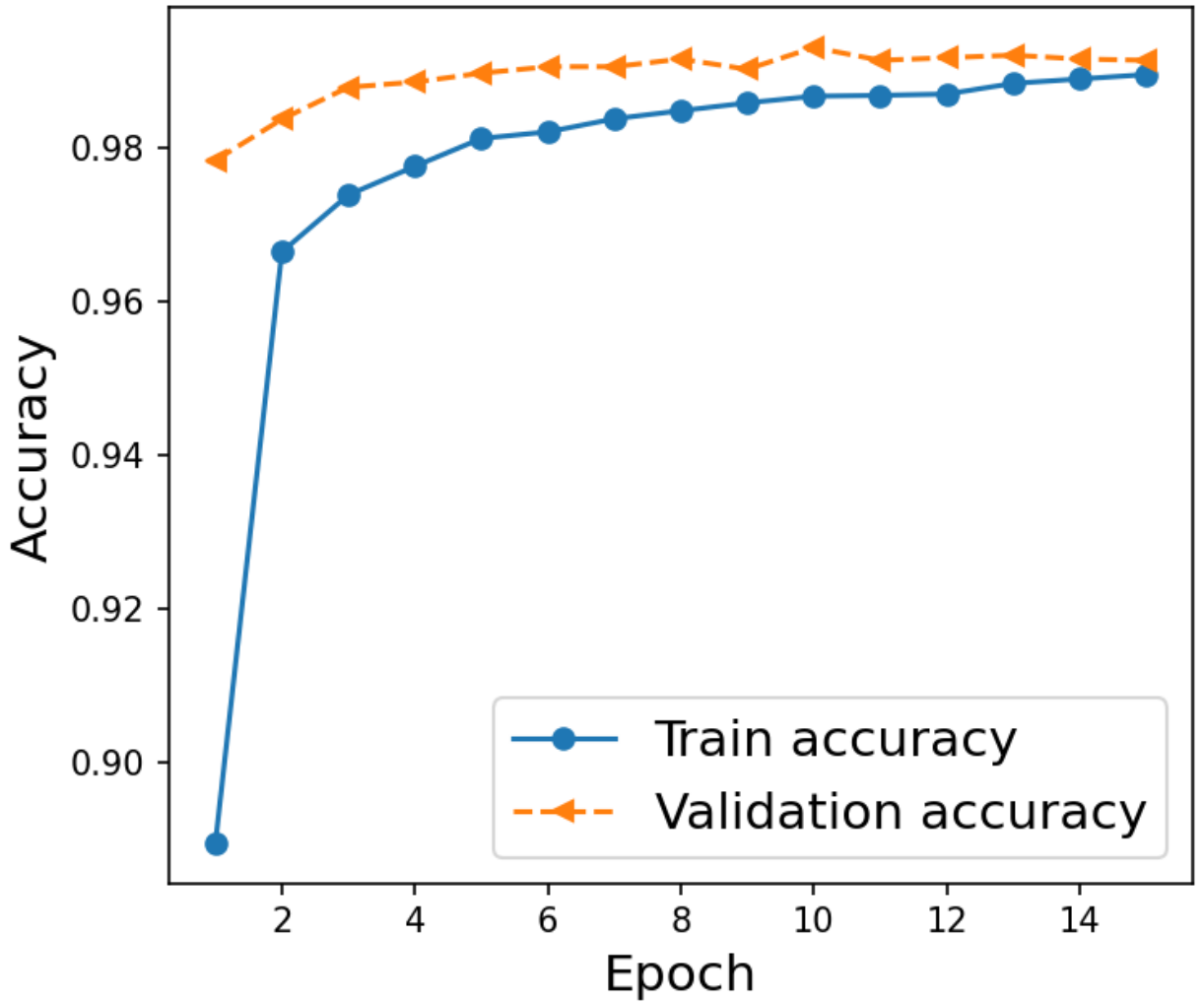
```
>>> history = model.fit(x_train, y_train, batch_size=128, epochs=15, validation_split=0.1)
Epoch 1/15
422/422 ━━━━━━━━━━━━━━━━━━━━ 7s 14ms/step - accuracy: 0.9891 - loss: 0.0325 - val_accuracy: 0.9932 - val_loss: 0.0282
Epoch 2/15
422/422 ━━━━━━━━━━━━━━━━━━━━ 5s 12ms/step - accuracy: 0.9905 - loss: 0.0290 - val_accuracy: 0.9913 - val_loss: 0.0301
Epoch 3/15
422/422 ━━━━━━━━━━━━━━━━━━━━ 5s 13ms/step - accuracy: 0.9896 - loss: 0.0299 - val_accuracy: 0.9920 - val_loss: 0.0303
Epoch 4/15
422/422 ━━━━━━━━━━━━━━━━━━━━ 6s 14ms/step - accuracy: 0.9895 - loss: 0.0291 - val_accuracy: 0.9925 - val_loss: 0.0294
Epoch 5/15
422/422 ━━━━━━━━━━━━━━━━━━━━ 7s 18ms/step - accuracy: 0.9914 - loss: 0.0251 - val_accuracy: 0.9932 - val_loss: 0.0285
Epoch 6/15
273/422 ━━━━━━━━━━━━━━━━━━━━ 2s 17ms/step - accuracy: 0.9909 - loss: 0.0272
```

```
#Plot results
hist = history.history
x_arr = np.arange(len(hist['loss'])) + 1
fig = plt.figure(figsize=(12,4))
ax = fig.add_subplot(1,2,1)
ax.plot(x_arr, hist['loss'],'-o',label ='Train loss')
ax.plot(x_arr,hist['val_loss'],'--<', label = 'Validation loss')
ax.set_xlabel('Epoch', size = 15)
ax.set_ylabel('Loss', size = 15)
ax.legend(fontsize = 15)
```

```
ax = fig.add_subplot(1,2,2)
ax.plot(x_arr, hist['accuracy'],'-o',label ='Train accuracy')
ax.plot(x_arr,hist['val_accuracy'],'--<', label = 'Validation accuracy')
ax.set_xlabel('Epoch', size = 15)
ax.set_ylabel('Accuracy', size = 15)
ax.legend(fontsize = 15)
```

```
plt.show()
```





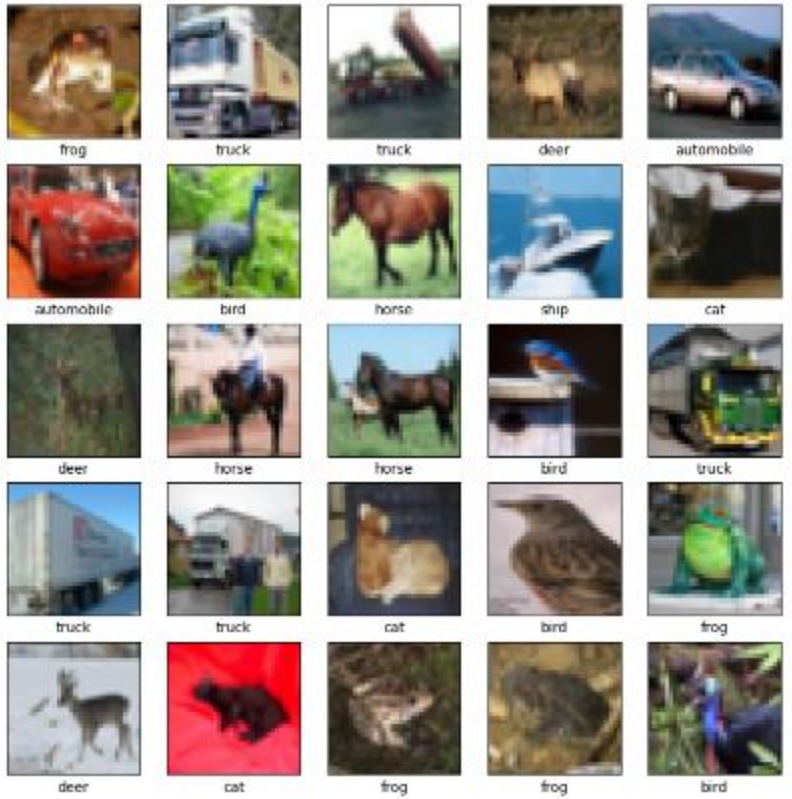
```
# Calculate accuracy
score = model.evaluate(x_test, y_test, verbose=0)
print("Test loss:", score[0])
print("Test accuracy:", score[1])
```

```
>>> print("Test loss:", score[0])
Test loss: 0.026052663102746 01
>>> print("Test accuracy:", score[1])
Test accuracy: 0.9912999868392944
```

To experiment with the MNIST dataset using different parameters, you would adjust the following settings and evaluate the model's performance using metrics like accuracy, precision, recall, and F1-score on the test dataset

Some problems with **CNNs**:

- **Feature extraction** from image data using **VGG16**, **VGG19**.
- **Object detection**: Using pre-trained CNN models (such as **YOLO** or **Faster R-CNN**) to detect objects in images or videos.
- **Face recognition**
- **Medical image analysis**
- **Natural Language Processing (NLP)**
- **Video analysis**: action recognition, video segmentation, and object tracking in videos.
- **Image generation**: Generative Adversarial Networks (**GANs**).



https://datafiction.github.io/docs